DEEP LEARNING

Text Sentiment Classification

Training, Evaluation, Saved Model, and Inference

Executive summary. This project implements a PyTorch multilayer perceptron (MLP) to classify short text reviews as positive or negative. It includes data preprocessing, stratified train/validation/test splits, training and validation curves, classification metrics, ROC analysis, saved model weights, and an inference script. A Logistic Regression reference is evaluated to contextualize the neural model.

1. Objective and dataset

The goal is to demonstrate an end-to-end deep-learning text classification workflow. The bundled CSV contains 370 balanced example reviews: 185 positive and 185 negative. It is a small curated educational corpus created for this project, not a public benchmark or representative sample of customer feedback.

2. Preprocessing and model design

Text is lowercased and tokenized by scikit-learn TF-IDF vectorization using word unigrams and bigrams. The vectorizer is fitted only on the training partition. Data was split stratified into 236 training, 60 validation, and 74 test examples (random seed 42). The PyTorch MLP contains a dense hidden layer with 32 units, ReLU activation, dropout (0.50), and a two-class output layer. Training uses cross-entropy loss and AdamW; the best validation-loss checkpoint is retained. A Logistic Regression model is fitted as a reference baseline.

3. Test results

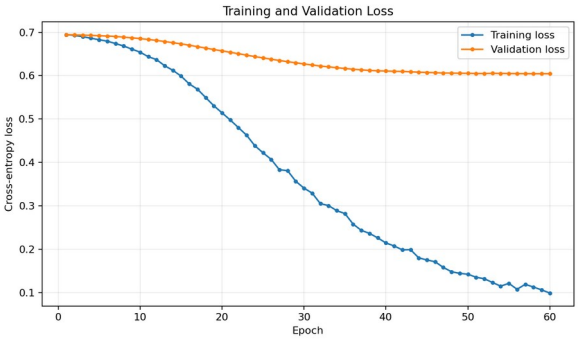
Model	Accuracy	Precision	Recall	F1-score	ROC-AUC
PyTorch MLP	0.6216	0.6154	0.6486	0.6316	0.6486
Logistic Regression reference	0.5541	0.5556	0.5405	0.5479	0.6844

Interpretation: the MLP achieved 62.16% accuracy and 0.6486 ROC-AUC on the held-out test set. The Logistic Regression reference had lower accuracy (55.41%) but a higher ROC-AUC (0.6844). Therefore, the neural model is not the best on every metric. The 74-row test set is small, so small changes in data splitting can change the scores substantially.

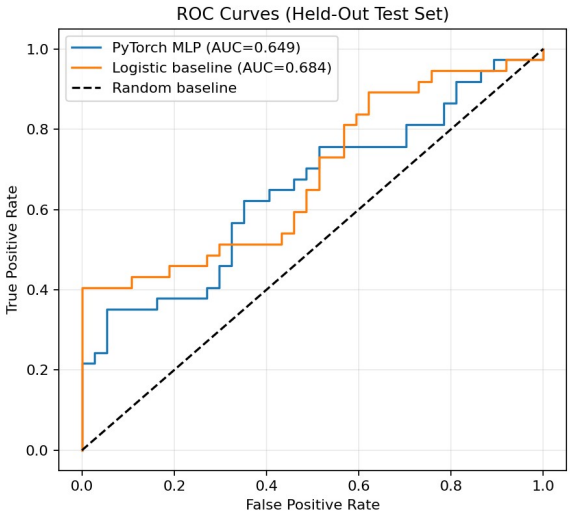
4. Visual evaluation

The loss plot tracks optimization over epochs; the confusion matrix shows test classification counts; the ROC curve compares ranking performance across thresholds; and the final chart compares ROC-AUC values for the MLP and Logistic Regression reference.

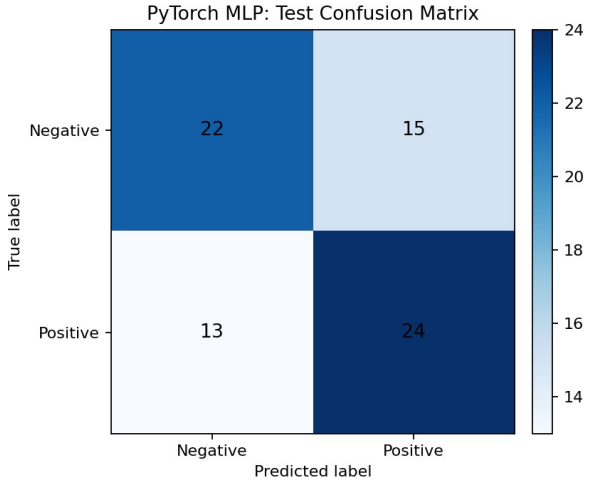
Training / validation loss



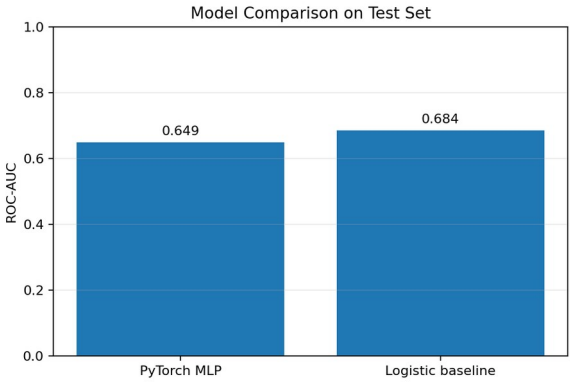
Test ROC curves



MLP confusion matrix



ROC-AUC model comparison



5. Saved model and how to run inference

The trained PyTorch weights and architecture settings are stored in `artifacts/sentiment_mlp.pth`. The fitted TF-IDF vectorizer is also required and stored in `artifacts/tfidf_vectorizer.joblib`. Run these commands from the project root after installing `requirements.txt`:

```
python train_sentiment.py --epochs 60 --batch-size 32 --seed 42 --patience 12
python predict_sentiment.py "The delivery was quick and the product works perfectly."
python predict_sentiment.py "The app crashes constantly and support did not help."
```

6. Transfer learning and limitations

An optional `train_distilbert_transfer.py` script is supplied to fine-tune pretrained DistilBERT when network access is available; downloading and fine-tuning that transformer is not part of the included trained artifact. The delivered, offline-capable saved model is the PyTorch MLP. Because the dataset is small and curated, the result may not generalize to slang, sarcasm, mixed sentiment, or real customer language. Softmax scores are not calibrated probabilities. A larger licensed dataset and further error analysis are required before any practical deployment.

Project files

Notebook: `notebooks/Deep_Learning_Sentiment_Classification.ipynb`
Training: `train_sentiment.py`
Inference: `predict_sentiment.py`
Optional transfer learning: `train_distilbert_transfer.py`
Environment and exact commands: `README.md`
Saved metrics and plots: `artifacts/`